

ラボ 2.3: EEPROM ダンプ内のパスワード

1. はじめに

1.1. 必要なリソース

- ControlThings プラットフォーム仮想マシン

1.2. 概要

ラボでは、メモリチップから取得したいいくつかのファイルと、メモリチップから取得したデータと同様のシミュレーションデータを分析します。これらのメモリダンプから暗号鍵とパスワードを探します

1.3. 目的

- xxdを使用してバイナリファイルを手動で解析する
- 文字列を使用してパスワードを識別する
- エントロピーグラフを使用して非対称鍵を見つける

2. ラボの手順

ControlThings Platform VMを起動する

ヒント

ICS410 Windows 10 VMをシャットダウンし、すべてのラボで一度に1つのVMのみを実行することをお勧めします。これにより、システムリソースを節約し、処理速度を向上できます。

2.1. XXDによるバイナリ解析

これらのファイルを分析するためにまず最初に行うことは、16進ダンプツールを使用してデータセットを手動で確認することです。ControlThings Platformの仮想マシンでは、この `xxd` ツールを使ってこれを行うことができます。さらに簡単にするために、出力をパイプ処理して、`xxd` 矢印 `less` キーで上下にスクロールできるようにします。

ターミナルを開く

画面左上の「アクティビティ」リンクをクリックしてメインメニューに移動し、画面下部にある黒いボックスのようなターミナルアイコンをクリックします。下部にターミナルボックスが表示されない場合は、上部の検索ボックスで「ターミナル」を検索してください

最初に見るファイルはです `memdump.bin`。

警告

以下のタスクでは、入力するテキストに加えて、コマンドプロンプトも表示されます。これは、正しいディレクトリにいることを確認するのに役立ちます。間違った場所にいるとプロンプトの表示が異なります。入力するテキストは `will look like this ...`

```
cd ~/Samples/Memory
```

最初の文字はチルダ文字であり、ダッシュやハイフンではありません。ほとんどの英語キーボードでは、チルダ文字は左上隅のタブキーの上にあり、使用するにはShiftキーが必要です。チルダ文字はホームディレクトリへのショートカットです。

米国英語以外のキーボードを使用しており、VM でキーボードの言語をまだ変更していない場合は、ラップトップのセットアップに関する Lab0 ドキュメントに戻って、変更方法の説明を参照してください。

```
xxd memdump.bin | less
```

コマンドの前の文字 `less` はパイプ文字です。ほとんどの英語キーボードでは、これは通常**Enter**キーの上にあり、使用するにはShiftキーが必要です。

✓ 成功

```
00000000: 3c9e 3192 c173 a079 22fd d8ea 571c 9b38  <.1..s.y"...W..8
00000010: 1d9a 2889 c5f6 c1a4 d4a2 a330 bc25 8938  ..(.....0.%.8
00000020: 8028 1ccd 5c6e 0e6c 30e9 8f15 0c67 f442  .(..\n.l0....g.B
00000030: dec8 772f 6fdf 9fdf 0dac 86af 97c3 66cc  ..w/o.....f.
00000040: c0a4 6496 1fd5 43b7 459f ea79 6fef 2102  ..d...C.E..yo.!
```

...出力は続きます...

コマンドを使用してこのファイルを `less` 表示しています。上下矢印キーを使用してファイルをスクロールできます。また、**PgUp** キーと**PgDn**キーも使用できます

? この出力をどのように読み取ればよいのでしょうか?

xxd ファイルの生の 16 進ダンプが 3 つの列に分かれて表示されます。

Address	Raw Hex Dump of File	ASCII Decode of File
00000000:	3c9e 3192 c173 a079 22fd d8ea 571c 9b38	<.1..s.y"...W..8
00000010:	1d9a 2889 c5f6 c1a4 d4a2 a330 bc25 8938	..(.....0.%.8
00000020:	8028 1ccd 5c6e 0e6c 30e9 8f15 0c67 f442	.(..\n.l0....g.B
00000030:	dec8 772f 6fdf 9fdf 0dac 86af 97c3 66cc	..w/o.....f.
00000040:	c0a4 6496 1fd5 43b7 459f ea79 6fef 2102	..d...C.E..yo.!.

? 人間が読めるパスワードはありますか?

明らかなパスワードは表示されないはずですが、試してみる価値は常にありますし、組み込みデバイスでよく見られるものでもあります。

? データに異常は見られますか?

アドレス周辺では、`00000bf0` 非印字文字の頻度が変化していることがわかります（ASCII セクションではピリオドで表示されます）。多くの行で、非印字文字が 50% 以上から印字文字が 100% に変化しています。これはかなり顕著です。

おめでとうございます！ バイナリ EEPROM ダンプから最初の暗号キーを見つけました。

あなたの背景や経験に応じて、この印刷可能な文字の大きなブロックは、暗号キーの保存で非常に一般的な手法である base64 でエンコードされていることに気付くでしょう。

? Base64 でエンコードされているかどうかはどうやってわかりますか?

= ほとんどの人は、またはで終わる ASCII ブロックの によりこれを認識します。これは、== 最初の文字数が 3 で割り切れない場合に Base64 がパディングとして使用します。

ただし、Base64 を認識するより良い方法は、ASCII ブロックが次の文字を完全に組み合わせて使用していることを認識することです。

```
* 0123456789 (10 characters)
* ABCDEFGHIJKLMNOPQRSTUVWXYZ (26 characters)
* abcdefghijklmnopqrstuvwxyz (26 characters)
* +/ (2 characters)
```

$10+26+26+2 = 64$ となる。

URL-Safe Base64 と呼ばれるこのバリエーションでは、+/文字が_文字に置き換えられます。

このキーについては、このクラスの範囲を超えており、より複雑な処理になる可能性があるため、これ以上は何も行いません。組み込みハードウェアで攻撃者がこれらのキーを見つけるのがいかに簡単か、ご理解いただけたかと思います。

このファイルの分析が終了したら `q` 終了するには、次のように入力します。 `less`

`ls` コマンドを使用してこのディレクトリ内のファイルを一覧表示します

とを使用し `xxd` で、 `less` このディレクトリ内の他のファイルに対して同様の分析を実行します。

2.2. 文字列による解析

バイナリデータセットを手動で分析する際に使用できるもう一つのツールは、 `strings` コマンドです。このコマンドを使用すると、印刷可能な文字のブロックのみに焦点を当て、印刷不可能な文字はすべて無視できます。印刷不可能な文字を無視することで一部の異常を見逃すこともあります、では見られない異常が見つかることもあります `xxd`。

```
strings memdump.bin | less
```

✓ 成功

矢印キーを使ってファイルの中央までスクロールすると、次のデータが表示されます

```
TM?<
}BD9Y&
LyBr
qmB,m
tLV'
x\`ig
\|q/p
]i`'
XnddmT6
MIICWwIBAAKBgQCx1kzejuLoL2mKzNEgMxKDEjiWrSg0pl1v9tI5ZFNzfp2H7A9KsSL+k7kY
pLPGqFVsv5U6sH/pkneeol0ralzHJBoGcIt0lUGl+Ac/BFv9JfLGhTh6iChMN/cxVU4BpHR8
G0/oEoWYoQ+qxKpY64jxwfe607AVoFIKmF1HeCbVvQIDAQABAn9aTni6iwFu5qSx11BbCZRC
+uPBoCzKj7DjKn0WsVuvSpy0aStawpkCrgFQgLLIjrLShndBVl+ePMKBy+yxH1H89ZPAXkG+
HGvXm3niq6V4vDYKE6rXLnR+VwdPCWtUy6m0l17t4o4AgB/8yww/F0FSrXECF4qwnTwPRJyM
lp/ZAkEA2qZ0tsONIZwD2vhFaZDSvKPfnb/D+AbL40fScMlFOUIaPmBIH4D7Fvw2E1GVhMHR
VGyCjj1ypHesfDTXqeJTDwJBANA3F72z/loeV0bFJK0Xr5FX0Kz44Dbu8PsKpagCCsPTKjdB
zLzLVJGz+MrhDfq2k40x2BqkTAfRKeuTIpyyB1sCQHsPFj9KviOngm/6LW7lsIW1IYTySgRN
JvFrGdianthEC0s0W0NH34mnN3QWBbXrM7pxk1yk3uIL7MsXHo4hsCcCQCsbkxqA43bvv70
LJNJ4cF9S+7b1u0Ifp5HTwOVlmoPshA8cDAWoInLNNc/7WqxaXlw4sACbI1z6eu7Wuy6mpGF
AkeAwKTcmLdC4UBW9RapbRHfJg3AWFy840uTPrET/bBwVKDMRG2r9G0zKSsuTStZdNgcqYqo
GbN2Br8tHRvRXqbh2A==
```

この `strings` コマンドは、出力結果を制御するためのオプションと組み合わせることができます。 `-n` オプションを使用すると、表示する文字列のサイズを選択できます。例えば、は文字列を検索して出力しますが、文字列は `-n 8` 検索して出力しません。

```
password passwd
```

```
strings -n 12 memdump.bin
```

✓ 成功

```
MIICWwIBAAKBgQCx1kzejULoL2mKzNEgMxKDEjiWrSg0pl1v9tI5ZFNzfp2H7A9KsSL+k7kY
pLP6qFVsv5U6sH/pkneeol0ralzHJBoGcIt0lUGl+Ac/BFv9JfLGHTh6iChMN/cxVU4BpHR8
GO/oEoWYo+qxKpY64jxwfe607AVoFIKmF1HeCbVQIDAQABAn9aTni6iwFu5qSx11BbCZRc
+uPBoCzKj7DjKnOWsVuvSpy0aStawpkCrgFQgLLIjrLShndBVl+ePMKBy+yxH1H89ZPAXkG+
HGVXm3niq6V4vDYKE6rXLnR+VwdPCWtUy6m0L17t4o4AgB/8yww/F0FSrXECF4qwnTwPRJyM
lp/ZAkEA2qZ0tsONIzWd2vhFaZDSvKPfmb/D+AbL40fScMLF0UIaPmBIH4D7Fvw2E1GVhMHR
VGYCjj1ypHesfDTXqeJTDwJBANA3F72z/loeV0bFJK0Xr5FX0Kz44Dbu8PsKpagCCsPTKjdB
zLzLVJGz+MrhDfq2k40x2BqkTafRKeuTIpyyB1sCQHsPFj9KviOngm/6LW7lsIW1IYTySgRN
JvFrGdianthEC0s0W0NH34mnN3QWBbXrM7pxk1yk3uIL7MsXHo4hsCcCQQCSbkxqA43bvv70
LJNJ4cF9S+7b1u0Ifp5HTwOVlmoPshA8cDAWoInLNNc/7WqxaXlw4sACbI1z6eu7Wuy6mpGF
AkEAwKTcmLdC4UBW9RapbRHfJg3AWFy840uTPrET/bBwVKDMRG2r9G0zKSsuTStZdNgcqYqo
GbN2Br8tHRvRXqbh2A==
```

これにより、12 文字未満の印刷可能な ASCII 文字列をすべて削除し、暗号キーだけを残すことができます。

コマンドを複数個連結し、あるコマンドの出力を別のツールで処理すると便利な場合があります。これは | パイプ文字を使用することで実現できます。この場合、最後の `strings` コマンドの出力を処理して、コマンドで文字列をデコードできます `base64 -d`。

```
strings -n 12 memdump.bin | base64 -d
```

✓ 成功

これは、生の暗号キー バイトの ASCII デコードを強制するため、一連の奇妙な文字が表示されます。

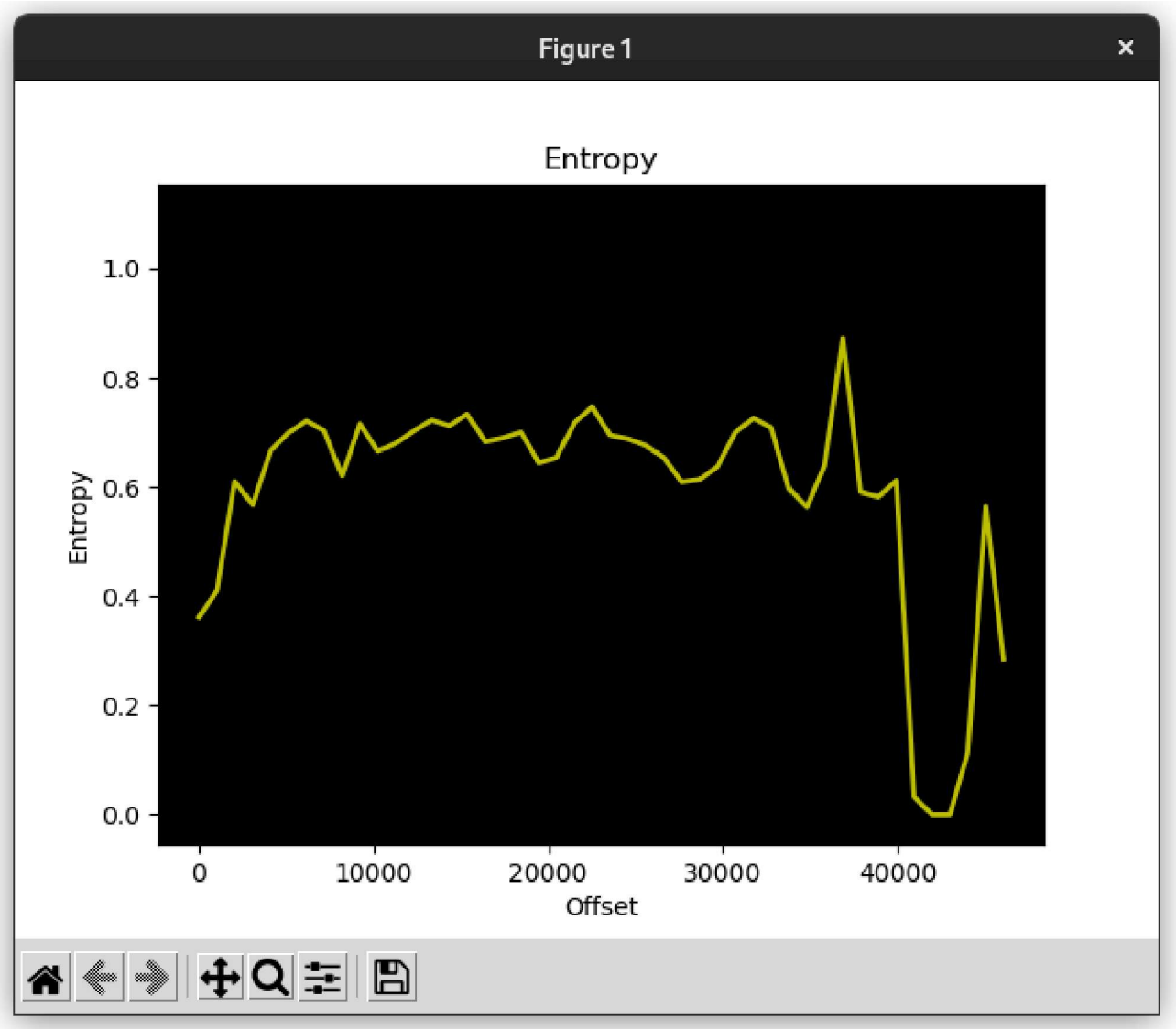
```
0[?L>B/i 38(4)o9dSs~?J?"?zUL?:wSj\?p?tA?
[%F8z(L71UNt|IX?;R
]Gx&UZNx?n複P[ \,v*s[?J?i+ZPpRÔwAV_<?Q?^A?eW?y=|x6
?.t~WO kT]?^?
Rq?<D?A?t?I#
<?Ei?e?d?G?p?E9B?>`H?6?Tf?=r?w?|4?SA?7?ZWf?$?Wb?6?
?
?L?)?"[?J?#?o?-n?!!?JM&?k?4[CG7?T?3?q?\?
?X\?K?>?pT?Dm?m3)+.M+Yt?v?-^?%
```

2.3. Binwalkによるエントロピー分析

ファイルを確認してください `hidden-key-raw`。このファイルに `xxd` を使っても `strings`、隠された暗号鍵を見つけることはできません。なぜなら、暗号鍵は周囲のデータと同じように見えるからです。しかし、エントロピーグラフを作成するツールを使ってデータセット内の高度にランダムなデータブロックを探すと、暗号鍵を見つけるのは容易になります。 `binwalk -E` このディレクトリ内の他のファイルにも、(エントロピーを表す `-E`) を使ってこのアプローチを試してみてください。

```
binwalk -E hidden-key-raw
```

✓ 成功



ターミナル出力：

DECIMAL	HEXADECIMAL	ENTROPY
0	0x0	Falling entropy edge (0.361548)
37888	0x9400	Falling entropy edge (0.591045)

ファイル `hidden-key-raw` 内の小さなデータブロックごとにエントロピー分析を行うことで、暗号鍵を見つけるのがはるかに簡単になります。暗号アーティファクトはランダムノイズに隠れるように設計されていますが、非ランダムデータでは目立つように目立ちます。スパイクが大きいほど、暗号アーティファクトが見つかった可能性が高くなりますが、必ずしも保証されるわけではありません。圧縮データやランダムデータもエントロピーが高いため、確実にするためにはさらなる分析が必要になることがよくあります

バイト オフセット 0x9000 から始まるスパイクは隠された暗号キーであり、`xxd` 必要に応じて見つけることができます。

`binwalk -B` 既知の暗号化キー ファイルのバイナリ署名を検索してキーを見つけることもできます。

binwalk -B hidden-key-raw

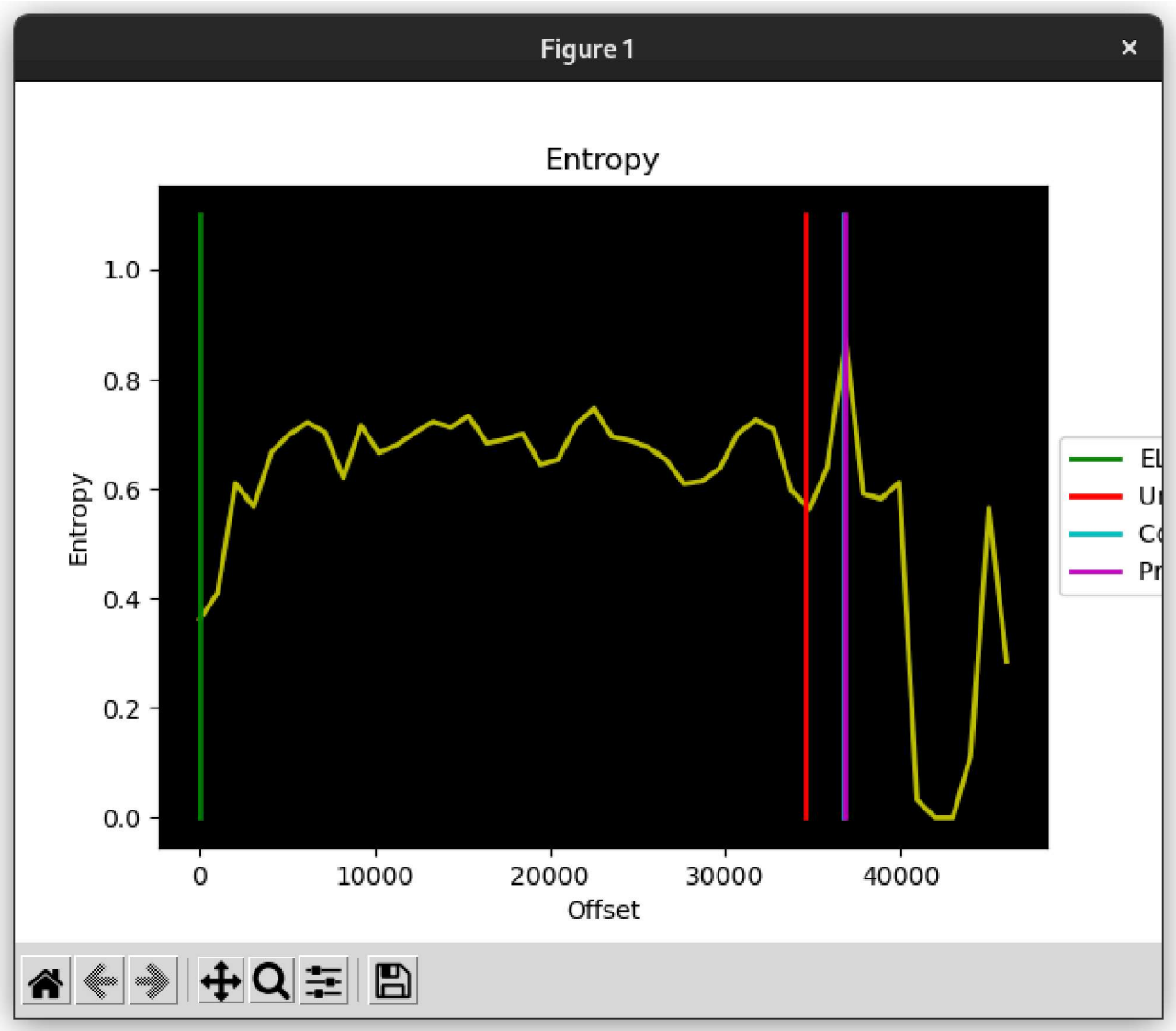
✓ 成功

DECIMAL	HEXADECIMAL	DESCRIPTION
0	0x0	ELF, 32-bit LSB executable, Intel 80386, version 1 (SYSV)
34564	0x8704	Unix path: /usr/share/locale
36768	0x8FA0	Copyright string: "Copyright %s %d Free Software Foundation, Inc."
36864	0x9000	Private key in DER format (PKCS header length: 4, sequence length: 603)

`Binwalk` これら 2 つの機能を組み合わせて、エントロピー グラフ上にマーカールを表示できます。

binwalk -EB hidden-key-raw

✓ 成功



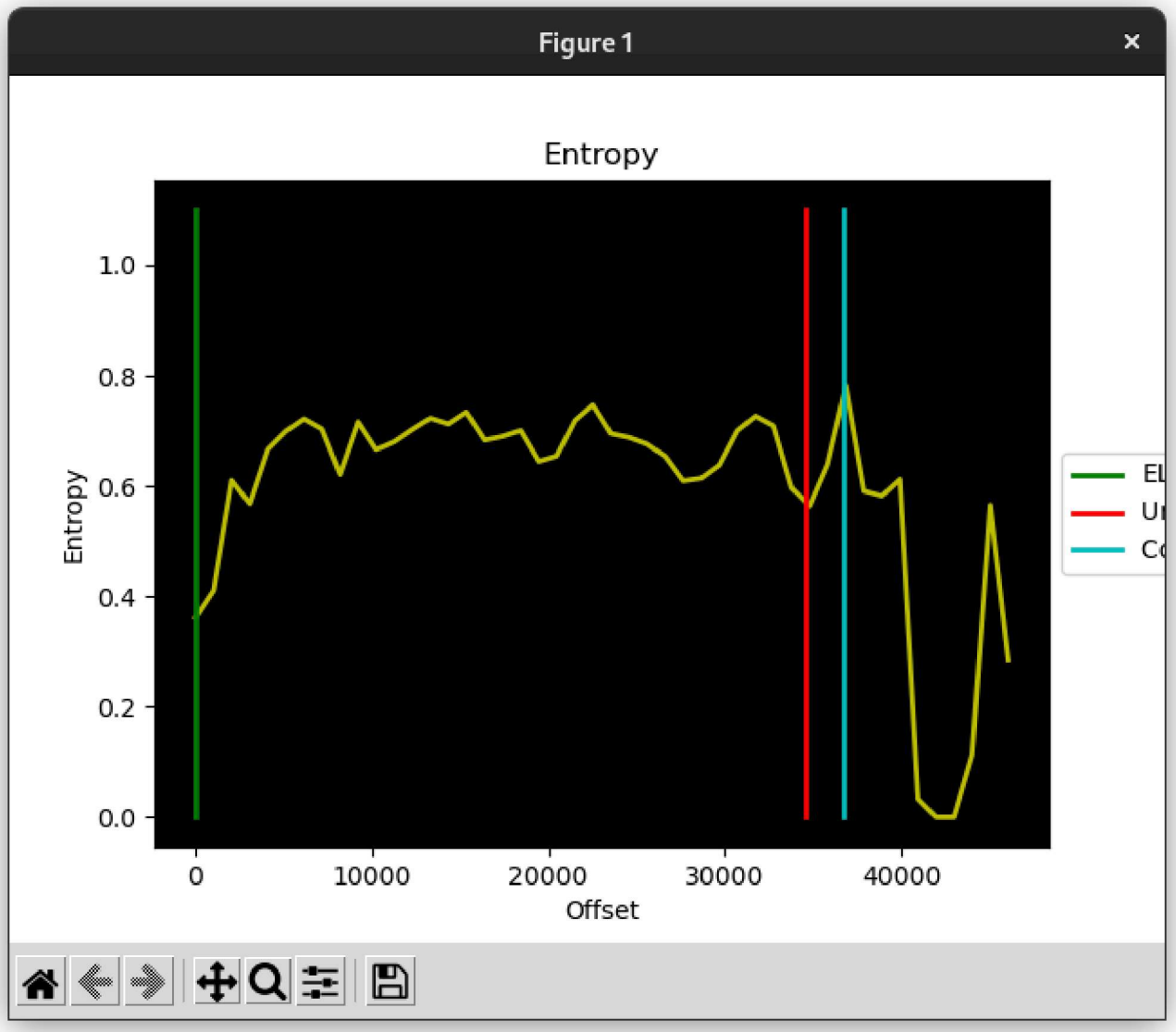
ターミナル出力：

DECIMAL	HEXADECIMAL	DESCRIPTION
0	0x0	ELF, 32-bit LSB executable, Intel 80386, version 1 (SYSV)
34564	0x8704	Unix path: /usr/share/locale
36768	0x8FA0	Copyright string: "Copyright %s %d Free Software Foundation, Inc."
36864	0x9000	Private key in DER format (PKCS header length: 4, sequence length: 603)

エントロピー法を用いて、生の形式ではなくBase64エンコードされた鍵を見つけることは困難ですが、それでも可能です。鍵をBase64エンコードするプロセスでは、鍵を表すために使用される文字数が制限されるため、相対的なエントロピーが低下します。また、BinwalkにはBase64エンコードされた鍵のバイナリ署名がないため、この点もさらなる欠点となります。

binwalk -EB hidden-key-base64

✓ 成功



スパイクがそれほど大きくなく、端末出力にキーが識別されていないことに注意してください。

ターミナル出力：

DECIMAL	HEXADECIMAL	DESCRIPTION
0	0x0	ELF, 32-bit LSB executable, Intel 80386, version 1 (SYSV)
34564	0x8704	Unix path: /usr/share/locale
36768	0x8FA0	Copyright string: "Copyright %s %d Free Software Foundation, Inc."

3. ボーナスチャレンジ

他の参加者のラボの完了を待っている場合、またはこのラボで紹介された概念をさらに深く探求したい場合は、以下のアイデアを試してみてください。これらのチャレンジにはチュートリアルは用意されていませんが、行き詰まった場合は、コース作成者のJustinに組みの成果を示せば、喜んでヒントを教えてください。justin [@controlthings.io](mailto:justin@controlthings.io)までメールでお問い合わせください

3.1. Hidden-Key-Base64

これらのファイルについては `hidden-key-*`、既存のLinux実行ファイルにDSA鍵を挿入しました。どの実行ファイルを使用してファイル `xxd` を作成したかを確認するために使用してください `hidden-key-*`